Middleware and Request Pipeline

LESSON 4

How every request travels through an ASP.NET Core application

1 What is a Middleware?

A small piece of code in the path of every request. It can do some work, **pass the request to the next** middleware, or **stop it** and respond itself.

2 What is the Pipeline?

An **ordered chain** of middlewares. The order is the order you write them in `Program.cs`. The request goes in, and the response comes back out in **reverse order**.

3 Use, Run and Map

Use A normal middleware. Calls `next()` to continue.

Run The **last** middleware. It has no `next`, so anything after it never runs.

Map **Branches** the pipeline for a specific path, like `/admin`.

4 Short-circuit

A middleware responds **without calling** `next()`. The request stops there and never reaches what comes after, like a gatekeeper rejecting it.

EXAMPLE

```
app.Use(async (context, next) =>
{
    Console.WriteLine("A: before next");
    await next();
    Console.WriteLine("A: after next");
});
```

With a second middleware B after it, the output is:

```
A: before next
B: before next
B: after next
A: after next
```

TYPICAL ORDER OF BUILT-IN MIDDLEWARES

1. `UseExceptionHandler`
2. `UseHttpsRedirection`
3. `UseStaticFiles`
4. `UseRouting`
5. `UseCors`
6. `UseAuthentication`
7. `UseAuthorization`
8. `Endpoints`

One request: Request → Middleware A → Middleware B → **Endpoint** → Response (back through B, then A)